FCT
FACULDADE DE CIÊNCIAS
E TECNOLOGIA
UNIVERSIDADE DOS AÇORES

Sistemas Distribuídos

Licenciatura em Informática

Luís Xavier

Miguel Medeiros

Sara Freire

1. Programa CAL_1:

1a. Funções e explicação “try...expect”:

Partindo do ficheiro original “CALCULADORA_1_ORIGINAL/main.py”, foi verificado que a função da raizquadrada estava incompleta, não executando nenhum cálculo. No entanto, as funções da multiplicação e da divisão estavam completas. As alterações feitas foram implementar a função raizquadrada e a utilização da biblioteca math.

Antes – Função raizquadrada incompleta (CALCULADORA_1_ORIGINAL/ main.py)

```
def raizquadrada( x: float) -> float:
    pass
```

Depois – Função Implementada (raiz_quadrada.py)

```
import math

def raiz_quadrada(a: float) -> float:  4 usages
    if a < 0:
        print("Erro: Não existe raiz de número negativo.")
        return 0.0
    return math.sqrt(a)
```

Foi utlizado a biblioteca math, que disponibiliza a função sqrt () para calcular raízes quadradas, com a verificação para números negativos, devolvendo 0.0 mostrando uma mensagem de erro, pois não é possível fazer uma raiz quadrada com números negativos dentro do conjunto de números reais.

Raiz Quadrada com números positivos:

```
Opções: +, -, *, /, v (para raiz quadrada)
Escolha a operação: v
Digite o número para a raiz: 49
Resultado: 7.0
```

Raiz Quadrada com números negativos:

```
Opções: +, -, *, /, v (para raiz quadrada)
Escolha a operação: v
Digite o número para a raiz: -9
Erro: Não existe raiz de número negativo.
Resultado: 0.0
```

Explicação do try...expect:

O bloco try...expect é um mecanismo de tratamento de exceções que permite ao programa reagir a erros em tempo de execução sem terminar repentinamente. No programa original, é usado na divisão:

```
try:
    z = dividir( x: 10,  y: 0)
    print("O resultado da soma é:", z)
except ZeroDivisionError as e:
    print("Error:", e)
```

O bloco try vai tentar executar o código, em caso de ocorrer um erro (neste caso ZeroDivisionError), o programa interrompe o try e executa o bloco expect, evitando que o programa "crashe". É sobretudo útil para situações previsíveis de erro como divisão por zero ou a entrada de valores não numéricos.

1b. Ficheiro main2.py – Todas as operações:

Foi criado o ficheiro main2.py que lê dois números do utilizador e apresenta automaticamente os resultados de todas as operações: soma, diferença, multiplicação, divisão e raiz quadrada, com as suas funções sendo importadas dos seus respetivos ficheiros.

```
from multiplicar import multiplicar
from somar import somar
from subtrair import subtrair
from raiz_quadrada import raiz_quadrada
from dividir import dividir
```

```
def iniciar_calculadora_total():   1 usage
    try:
        num1 = float(input("Digite o primeiro número: "))
        num2 = float(input("Digite o segundo número: "))

        print("\n--- Resultados ---")

        res_soma = somar(num1, num2)
        print(f"Soma ({num1} + {num2}) = {res_soma}")

        res_sub = subtrair(num1, num2)
        print(f"Diferença ({num1} - {num2}) = {res_sub}")

        res_mult = multiplicar(num1, num2)
        print(f"Multiplicação ({num1} * {num2}) = {res_mult}")
```

```python
        try:
            res_div = dividir(num1, num2)
            print(f"Divisão ({num1} / {num2}) = {res_div}")
        except ZeroDivisionError:
            print("Divisão: Erro - Não é possível dividir por zero.")

        try:
            res_raiz = raiz_quadrada(num1)
            print(f"Raiz Quadrada de {num1} = {res_raiz}")
        except ValueError:
            print(f"Raiz Quadrada: Erro - Não existe raiz real de número negativo.")

    except ValueError:
        print("Erro: Por favor, introduza apenas números válidos.")

if __name__ == "__main__":
    iniciar_calculadora_total()
```

- Resultados da execução do programa:

```
Digite o primeiro número: 16
Digite o segundo número: 9

--- Resultados ---
Soma (16.0 + 9.0) = 25.0
Diferença (16.0 - 9.0) = 7.0
Multiplicação (16.0 * 9.0) = 144.0
Divisão (16.0 / 9.0) = 1.7777777777777777
Raiz Quadrada de 16.0 = 4.0
```

- Resultados da execução do programa:

```
Digite o primeiro número: -12
Digite o segundo número: 7

--- Resultados ---
Soma (-12.0 + 7.0) = -5.0
Diferença (-12.0 - 7.0) = -19.0
Multiplicação (-12.0 * 7.0) = -84.0
Divisão (-12.0 / 7.0) = -1.7142857142857142
Erro: Não existe raiz de número negativo.
Raiz Quadrada de -12.0 = 0.0
```

- Resultados da execução do programa:

```
Digite o primeiro número: 16
Digite o segundo número: 0

--- Resultados ---
Soma (16.0 + 0.0) = 16.0
Diferença (16.0 - 0.0) = 16.0
Multiplicação (16.0 * 0.0) = 0.0
Divisão (16.0 / 0.0) = 0.0
Raiz Quadrada de 16.0 = 4.0
```

1c. Separação das funções em ficheiros individuais:

Cada função foi colocada no seu respetivo ficheiro Python separados, fazendo o código mais organizado. Foram criados os seguintes ficheiros:

- somar.py

```python
def somar(a: float, b: float) -> float:
    return a + b
```

- subtrair.py

```python
def subtrair(a: float, b: float) -> float:
    return a - b
```

- multiplicar.py

```python
def multiplicar(a: float, b: float) -> float:
    return a * b
```

- dividir.py

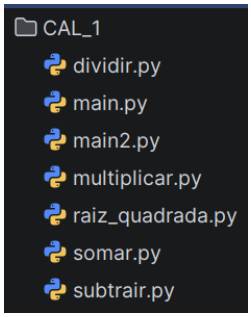
```python
def dividir(a: float, b: float) -> float
    if b == 0:
        return 0.0
    return a / b
```

- raiz_quadrada.py

```python
import math

def raiz_quadrada(a: float) -> float:  4 usages
    if a < 0:
        print("Erro: Não existe raiz de número negativo.")
        return 0.0
    return math.sqrt(a)
```

Estrutura final dos ficheiros da pastaCAL_1:



2. Programa CAL_2:

Partindo do ficheiro original CALCULADORA_2_ORIGINAL, identificou-se várias limitações e realizou-se um conjunto de alterações estruturais. A versão original

apenas suportava soma e divisão e o código das classes misturava a receção dos valores (no construtor) com o cálculo (no método executar ()). Já em CAL_2 reorganizou-se esta lógica e acrescentou-se as operações em falta.

2a. Remoção do construtor e nova interface calcular (num1, num2)

Na CALCULADORA_2_ORIGINAL, cada classe tinha um construtor __init__ que recebia os dois valores e os guardava como atributos e um método executar () sem parâmetros que usava esses atributos. Já na nossa versão, eliminámos o construtor e passámos os valores diretamente ao método, que remoamos para "calcular (num1, num2)". Desta forma, objeto não precisa de guarda estado entre chamadas, tornando a classe mais simples e reutilizável.

Antes – Classe Somar com construtor (CALCULADORA_2_ORIGINAL/somar.py)

```
class Somar:  1 usage
    def __init__(self, x:float, y:float):
        self.x = x
        self.y = y
        self.res = 0


    def executar(self)->float:
        self.res = self.x + self.y
        return self.res
```

Depois – Classe Somar sem construtor (Operacoes/somar.py)

```
class Somar:  3 usages
    """
    Classe responsável pela operação de soma.
    """
    def calcular(self, num1, num2):  2 usages
        """
        Retorna a soma de dois números.
        """
        return num1 + num2
```

2b. Classe Dividir – substituição de try/expect por validação condicional

Na CALCULADORA_2_ORIGINAL, a classe Dividir usava um bloco try...expect ZeroDivisionError, já na nova versão substituímos este mecanismo por uma verificação condicional explícita (if num2 == 0), que é mais legível.

Antes - Dividir com try/except (CALCULADORA_2_ORIGINAL/dividir.py):

```python
def executar(self)->Union[float,str]:
    try:
        self.res = self.x / self.y
    except ZeroDivisionError:
        return "error:dividing by zero"
    return self.res
```

DEPOIS — Dividir com validação condicional (Operacoes/dividir.py):

```python
class Dividir:  3 usages
    """
    Classe responsável pela operação de divisão.
    """
    def calcular(self, num1, num2):  2 usages
        """
        Retorna a divisão de num1 por num2.
        Retorna uma mensagem de erro se num2 for zero.
        """
        if num2 == 0:
            return "Erro: Divisão por zero"
        return num1 / num2
```

2c. Adição das classes Multiplicar e RaizQuadrada

A CALCULADORA_2_ORIGINAL apenas implementava as operações de soma e divisão. Enquanto na nossa versão tem, a mais, as classes Multiplicar e RaizQuadrada, que seguem a mesma interface calcular (num1, num2). A classe RaizQuadrada opera apenas sobre num1.

Classe Multiplicar (Operacoes/multiplicar.py):

```python
class Multiplicar:  1 usage
    """
    Classe responsável pela operação de multiplicação.
    """
    def calcular(self, num1, num2):  1 usage
        """
        Retorna o produto de dois números.
        """
        return num1 * num2
```

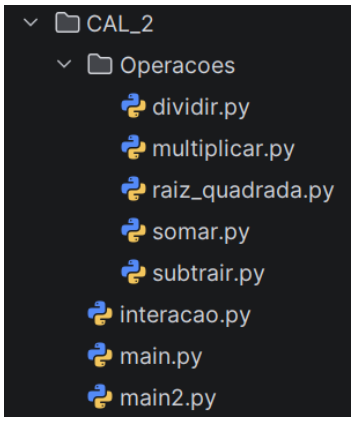
Classe RaizQuadrada (Operacoes/raiz_quadrada.py):

```python
import math

class RaizQuadrada:  1 usage
    """
    Classe responsável pela operação de raiz quadrada.
    """
    def calcular(self, num1, num2=None):  1 usage
        """
        Retorna a raiz quadrada do primeiro número.
        O segundo argumento (num2) é ignorado, mantido apenas para
        compatibilidade com a chamada padrão das outras operações.
        """
        if num1 < 0:
            return "Erro (Número negativo)"
        return math.sqrt(num1)
```

2d. Reorganização dos ficheiros em pasta Operacoes

Na CALCULADORA_2_ORIGINAL, os ficheiros somar.py, subtrair.py e dividir.py estavam na raiz da pasta do projeto, ao lado do main.py. Na nossa versão, movemos todas as classes para uma pasta dedicada chamada Operacoes, o que organiza melhor o projeto e separa claramente o código do programa principal do código das operações.



Execução main.py:

```
CALCULADORA
Opções: +, -, /
Escolha a operação: /
Número 1: 12
Número 2: 45
Resultado: 0.26666666666666666
```

2e. Reescrita do main.py e criação do main2.py

O main.py da CALCULADORA_2_ORIGINAL era uma função simples (main ()) com lógica de menu escrita diretamente com if/elif. Em contrapartida na nossa versão, reescrevemos o main.py encapsulando toda a lógica numa classe InterfaceCalculadora, com métodos separados para exibir o menu (exibir_menu ()), ler os inputs do utilizador (obter_inputs ()) e controlar o fluxo principal (iniciar ()). As instâncias das classes de operação são guardadas num dicionário (op_map), o que torna trivial adicionar novas operações no futuro.

ANTES: CALCULADORA_2_ORIGINAL/main.py

```python
import somar
import dividir
#import Operacoes.dividir as dividir
import interface
def main():  1 usage
    print("Qual é o cálculo que quer efetuar? x + - /")
    res:str = input()
    print("Preciso que introduza dois valores:")
    x:float = float(input("x="))
    y:float = float(input("y="))
    if res =="+":
        s:object = somar.Somar(x,y)
        res = s.executar()
        print("O valor da operação somar é:", res)
    elif res =="/":
        s:object = dividir.Dividir(x,y)
        res = s.executar()
        if type(res)==str:
            print (res)
        else:
            print("O valor da operação divisão é:",res)

if __name__ == '__main__':
    main()
```

DEPOIS: CAL_2/main.py

```python
class InterfaceCalculadora:  1 usage
    def __init__(self):
        self.op_map = {
            '+': Somar(),
            '-': Subtrair(),
            '/': Dividir()
        }

    def exibir_menu(self):  1 usage
        print("\nCALCULADORA")
        print("Opções: +, -, /")

    def obter_inputs(self, operador):  1 usage
        try:
                num1 = float(input("Número 1: "))
                num2 = float(input("Número 2: "))
                return (num1, num2)
        except ValueError:
            print("Erro: Por favor, digite apenas números válidos.")
            return (None, None)
```

```python
    def iniciar(self):  1 usage
        self.exibir_menu()
        operador = input("Escolha a operação: ")
        if operador in self.op_map:
            operacao_obj = self.op_map[operador]

            num1, num2 = self.obter_inputs(operador)

            if num1 is not None:
                resultado = operacao_obj.calcular(num1, num2)
            print(f"Resultado: {resultado}")
        else:
            print("Operação desconhecida.")

if __name__ == "__main__":
    app = InterfaceCalculadora()
    app.iniciar()
```

O uso do dicionário op_map elimina completamente a necessidade de blocos if/elif para cada operação: basta fazer op_map[operador] para obter o objeto correto. Isto significa que adicionar uma nova operação no futuro requer apenas criar a classe correspondente e acrescentar uma entrada ao dicionário, sem alterar mais nenhum código. Também foi criado um ficheiro main2.py que delega toda a lógica para uma classe Interacao (definida em interacao.py). Esta separação vai ainda mais longe na organização: o main2.py torna-se um ponto de entrada mínimo, e toda a lógica de interação com o utilizador fica encapsulada na classe Interacao.

CAL_2/main2.py

```python
from interacao import Interacao

if __name__ == "__main__":
    app = Interacao()
    app.execute()
```

Execução de main.py da CAL_2 com as operações +, - e /

Exemplo:

```
CALCULADORA
Opções: +, -, /
Escolha a operação: -
Número 1: 12
Número 2: -6
Resultado: 18.0
```

Execução de main2.py via classe Interacao

Exemplo:

```
--- CALCULADORA 2 ---
Introduza o primeiro número: 16
Introduza o segundo número: 3
Valor da soma: 19.0
Valor da Substração: 13.0
Valor da Multiplicação: 48.0
Valor da Divisão: 5.333333333333333
Valor da Raiz (Num 1): 4.0
```